AI Document Intelligence System using Retrieval-Augmented Generation (RAG)

This report describes the design and implementation of an AI-powered document analysis system based on Retrieval-Augmented Generation (RAG). The system allows users to upload documents such as HR policies, employee contracts, and loan applications, and then ask questions about them. Instead of relying only on a language model's general knowledge, the system retrieves relevant sections from uploaded documents and uses them as context for generating accurate answers.

1. Problem Statement

Organizations manage large volumes of internal documents such as HR policies, financial records, loan applications, compliance documents, and employee contracts. Manually reviewing these documents is time consuming, inefficient, and prone to human error. Employees or analysts often spend significant time searching for specific information within large files.

An intelligent system capable of understanding documents and answering queries can significantly improve efficiency and reduce manual workload.

2. Project Objective

- Allow users to upload documents such as PDFs or text files.
- Automatically process and understand the document content.
- Store document embeddings in a vector database.
- Enable natural language querying of uploaded documents.
- Generate accurate answers grounded in document content.

3. System Architecture

The system follows a modular architecture consisting of four main components: Frontend Interface, Backend API Server, Vector Database, and Large Language Model integration.

Component	Purpose
Frontend (React / Next.js)	User interface for document upload and chat interaction
Backend (FastAPI)	Handles API requests and orchestrates the RAG pipeline
Vector Database (FAISS)	Stores document embeddings for similarity search
Relational Database (PostgreSQL)	Stores user data, document metadata, and chat history
LLM API	Generates final answers based on retrieved document context

4. System Workflow

- User uploads a document through the frontend interface.
- Backend extracts text from the document.
- The text is split into smaller chunks.
- Each chunk is converted into vector embeddings using an embedding model.
- Embeddings are stored inside a FAISS vector database.
- When the user asks a question, the query is converted into an embedding.
- The vector database retrieves the most relevant document chunks.
- The retrieved context and user query are sent to the Large Language Model.
- The LLM generates a context-aware answer.

5. Technology Stack

Layer	Technology
Frontend	React / Next.js
Backend	FastAPI (Python)
Vector Database	FAISS
Relational Database	PostgreSQL
AI Model	Large Language Model (LLM)
Embeddings	Sentence Transformers / OpenAI embeddings
Document Processing	Python libraries for PDF and text processing

6. Use Cases

- HR policy assistance for employees
- Loan document analysis in banking
- Legal contract analysis
- Customer support knowledge base
- Insurance claim document review
- Enterprise internal search systems

7. Future Enhancements

- Real-time document ingestion pipeline
- Cloud deployment using AWS or Azure
- Role-based access control for sensitive documents
- Multi-language support
- Advanced document summarization
- Microservices architecture for scalability

- Improved security and encryption

8. Conclusion

This project demonstrates how Retrieval-Augmented Generation can be used to build intelligent document analysis systems. By combining document retrieval techniques with Large Language Models, the system can provide accurate and context-aware answers based on internal documents. Such systems can significantly improve efficiency in industries such as HR, finance, legal services, and enterprise knowledge management.